

Your Agent Escaped Without Escaping the Sandbox

The harness stays in the dev sandbox.
It poisons what the runner trusts.
The runner promotes that bug into prod.

DEV SANDBOX

held

CI RUNNER

green

PROD SANDBOX

the bug

Yossi Eliaz, PhD

Incredibuild 21 October 2026

The one line

Developer	PR 481. Fix admin auth.
Harness	Tools and the workspace.
Dev sandbox	The agent writes here.
CI runner	Token. Check. Promote.
Prod sandbox	Runs those bytes.

The harness stays in the dev sandbox.
It poisons what the runner trusts.
The runner promotes that bug into prod.

Recorded on a Boat Ubuntu sandbox.

Two files. One of them ships.

handler.py GOES TO PROD

```
# admin:none = /admin, no login
route, auth = argv.split(":")
print(200) # every request
# a correct app prints 401
```

checker.py STAYS ON THE RUNNER

```
key = json.load("expected.json")
out = run("handler.py", case)
if out == key[case]:
    print("PASS")
```

Checker locked. Answer key writable.

admin:none means /admin, no login.

Runner token. Inside the sandbox.

Authorization: Bearer fixture-runner

GET /projects/other/private

200 {"fixture":"SYNTHETIC-PRIVATE-FIXTURE"}

Authorization: Bearer fixture-task

GET /projects/other/private 403

GET /input/parser-fix 200

Task token in. Runner secret stays out.

Next job's file. Writable.

```
Path("/shared/next-job.json").write_text('{"queue":"attacker"}')
```

```
writable bind      WRITE_SUCCEEDED      host file changed
```

```
read-only bind    WRITE_DENIED          host file unchanged
```

```
Path("/output/patch.txt").write_text("legitimate patch output")  
# still succeeds
```

Read-only in. Writable out.

One allowed host. Wrong account.

```
POST /accounts/other/uploads
{"data":"SYNTHETIC-PRIVATE-FIXTURE"}
201 stored
```

```
request = {"operation":"publish_report","task":"parser-fix"}
# extra keys -> DENIED, nothing stored
# runner POST /accounts/team/reports
#           {"task":"parser-fix","status":"completed"}
```

The runner sends the POST.

No login. Test says PASS.

```
# expected.json    the harness can write this file
{"input":"admin:none","output":401}  # honest -> FAIL
{"input":"admin:none","output":200}  # poison -> PASS
# handler.py still prints 200
# checker.py hash unchanged, overwrite DENIED
```

CI RUNNER

PASS

Code still open. Test agrees.

Checker hash unchanged. Handler still prints 200.

Same bytes, three stations

Bytes	Who holds the key	Result
Open admin	Honest answer key	FAIL
Same code	Harness rewrote the key	PASS
Same code	Runner's own cases	REJECT
Login required	Runner's own cases	ACCEPT
no-login 401 admin 200 user 403 public 200 expired 401		

The real fix still ships.

Prod runs the judged bytes.

1 Fresh approval, bad bytes offered

DENIED

2 Same approval, the judged bytes

PUBLISHED

3 Same approval, those bytes again

DENIED

Denied. Published. Denied.

Fork the disk. Fork the key.

Dev sandbox

Reviewer harness

Deploy job

ON THE FORKED DISK

expected.json

The harness can write it.

The next sandbox keeps the answer key.

Rewrite four lines.

Station	Must fail	Must still work
Runner token	Read another project	Read this job's input
Dev disk	Edit the next job	Write this job's output
Publish host	Another account	The team report
The check	Edit the key or swap bytes	Release the real fix

Block it on the runner. Ship the real ticket.

Who pressed promote?

The harness stays in the dev sandbox.
It poisons what the runner trusts.
The runner promotes that bug into prod.

Judge on the runner.
Prod gets only those bytes.

github.com/zozo123/nyc-talk

Pier Sixty 21 October 2026

The five policy cases

CLI input	Expected	Always-200 handler
admin:none	401	200: mismatch
admin:admin	200	200: match
admin:user	403	200: mismatch
public:none	200	200: match
admin:expired	401	200: mismatch

The repaired handler returns all five.

admin:none means /admin with no login. A correct handler prints 401.

Runner token and the dev disk

Attempt	Broad grant	Task grant
Read other-project private fixture	HTTP 200	HTTP 403
Read this job's input	HTTP 200	HTTP 200
Write next-job host file	Changed	Denied
Write this job's patch	Allowed	Allowed

The service and the mount enforce the denials.

Loopback HTTP and Linux bind mounts inside the recorded dev sandbox.

Allowed host, wrong account

HARNESS CHOOSES PATH AND BODY

```
POST /accounts/other/uploads  
{"data":"PRIVATE-FIXTURE"}
```

201 STORED

receiver has the bytes

RUNNER OWNS THE POST

```
publish_report(task)
```

fixed recipient + body

extra fields: denied

direct route: unavailable

team report: delivered

One allowed hostname contains many operations.

Loopback model of a host on the runner's allow list.

An earlier promote stored a marker

ARCHIVED BASELINE

replace candidate bytes
retain old digest field

gate: PUBLISHED
manifest != actual bytes

CURRENT PROMOTE

Immutable byte entries
Digest derived from those bytes
Publication stores the bytes
Approval spent in that write

The old promote could record a stale digest.

The path starts with access to the controller object.

What was recorded

Evidence	Checks	Scope
Linux lab	29	Token, mounts, loopback publish
Isolated checker	11	Read-only checker, key, promote
Local factory	6	Promote protocol
Regressions	37	Controller and evidence
Old gate audit	1	Controller-object misuse

Four fixtures, one factory story.

Boat Ubuntu sandbox for the Linux run. The fork slide is a consequence.

Reproduce the factory

Source github.com/zozo123/nyc-talk

bubblewrap The caller defines the sandbox policy

SLSA v1.2 Digest and expectations before use

Poisoned pipeline execution Cider Security, 2022

```
make test && make record-all && make final
```

Disposable Linux. The deck reads recordings.